CIP-B101

LABORATORY REPORT

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Student Name	Usieme Blessing Onome
Registration Number	
Course Code	CIP-B101
Course Title	Basic Computer Skills for Digital Forensics
Lab Title	Lab 1A: Number Systems, ASCII, Timestamps and Data Representation
Instructor Name	Mr. Aminu Idris, AMCPN
Platform Used	Kali Linux
Date of Submission	7th November, 2026

Introduction

This lab introduced basic computer concepts used in digital forensics. I practised converting number systems, reading text from hexadecimal values, converting timestamps, checking file hashes and understanding byte order. I used Kali Linux commands and Python to examine the examples.

Task A: Number Systems and Manual Conversion

Binary is base 2, decimal is base 10 and hexadecimal is base 16. Computers use binary to store information. Hexadecimal makes long binary values easier for people to read.

A1. Manual conversions

1. Convert 101010_2 to decimal:

$(1 \times 32) + (0 \times 16) + (1 \times 8) + (0 \times 4) + (1 \times 2) + (0 \times 1) = 32 + 8 + 2 = 42$.

Answer: 42_{10}.

2. Convert $AB2_{16}$ to decimal:

A = 10 and B = 11. $(10 \times 256) + (11 \times 16) + (2 \times 1) = 2738$.

Answer: 2738_{10}.

3. Convert 126_{10} to binary by dividing by 2. The remainders, read from bottom to top, are 1111110.

2	126		
2	63	r	0
2	31	r	1
2	15	r	1
2	7	r	1
2	3	r	1
2	1	r	1
2	0	r	1

Answer: 1111110_2.

4. Convert 2738_{10} to hexadecimal by dividing by 16. The remainders from bottom to top are A, B and 2.

16	2738		
16	171	r	2
16	10	r	11 (B)
16	0	r	10 (A)

Answer: $AB2_{16}$.

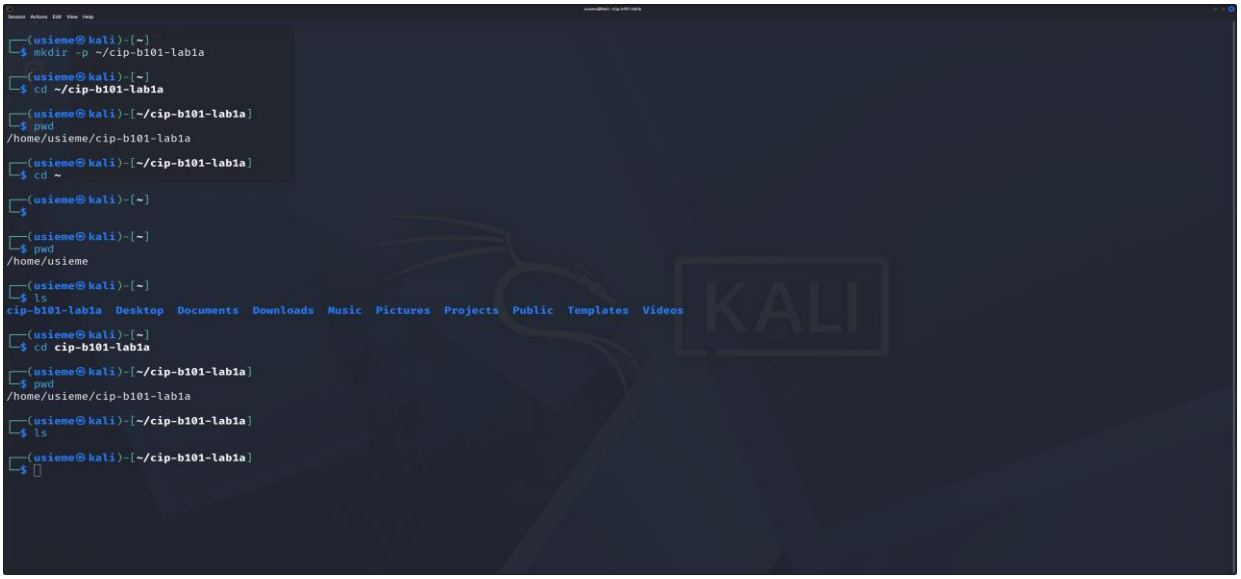
5. Group 101010110010_2 into groups of four from the right: 1010 1011 0010. These groups represent A, B and 2.

Answer: $AB2_{16}$.

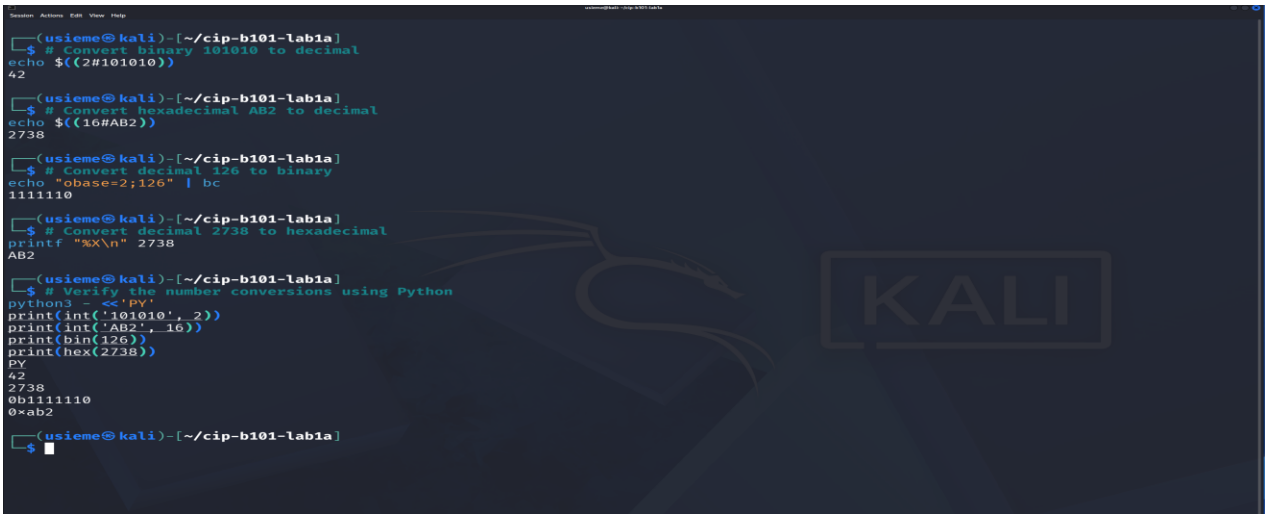
A2. Command validation

Value checked	Result	Command/tool used
101010_2 to decimal	42	echo $((2#101010)) - Bash arithmetic
$AB2_{16}$ to decimal	2738	echo $((16#AB2)) - Bash arithmetic
126_{10} to binary	1111110	echo "obase=2;126" | bc
2738_{10} to hexadecimal	AB2	printf "%X\n" 2738
Python cross-check	42, 2738, 0b1111110, 0xab2	Python3

The command results agreed with my manual calculations. The prefixes 0b and 0x in Python indicate binary and hexadecimal values.



Screenshot 1: Working Folder creation and pwd output



Screenshot 2: Binary/hex/decimal conversion commands

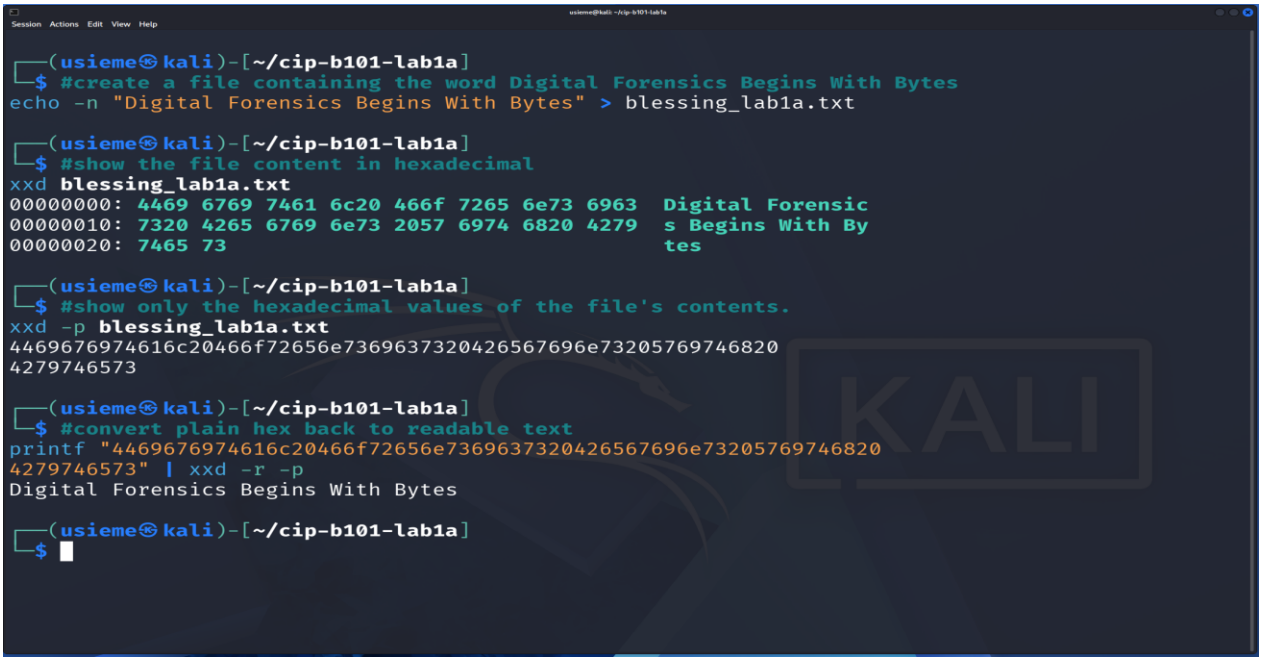
Task A result: The five conversions were completed and checked using Linux commands and Python.

Task B: ASCII, UTF-8 and Hex Evidence

A byte is a group of eight bits. ASCII assigns numbers to common characters. UTF-8 can represent many more characters and is compatible with standard ASCII characters. Hexadecimal is a convenient way to display byte values.

B1. Text file and hexadecimal output

I created a file named blessing_lab1a.txt containing the sentence: “Digital Forensics Begins With Bytes.” I used cat to display the text and xxd to view its hexadecimal bytes and readable character column. I also used xxd -p to display only the hexadecimal values.



Screenshot 3: xxd output for blessing_lab1a.txt

B2. Convert hexadecimal to text

I used the hexadecimal value 48656c6c6f and converted it back to text using xxd -r -p.

Decoded text: Hello.



Screenshot 4: xxd output for test evidence and Hex-to-ASCII conversion output

B3. The 0a newline byte

When echo is used without -n, it normally adds a newline at the end of the text. The newline is represented by the hexadecimal byte 0a. The -n option prevents echo from adding that extra newline.

This matters because the newline is part of the file's bytes, even if it is not obvious when reading the text on the screen.

B4. Why hexadecimal is not always readable text

An examiner should not assume that every hexadecimal value represents readable text. Bytes can also represent numbers, file structures, images, compressed data or encrypted information. The examiner must understand the type of data before trying to decode it as text.

Task B result: I used xxd to inspect bytes and converted a hexadecimal string back into readable text.

Task C: Epoch Time and Forensic Timestamps

A timestamp can help show when an event happened. In digital forensics, it is important to identify whether a time is UTC or local time so that events are not placed at the wrong time in a timeline.

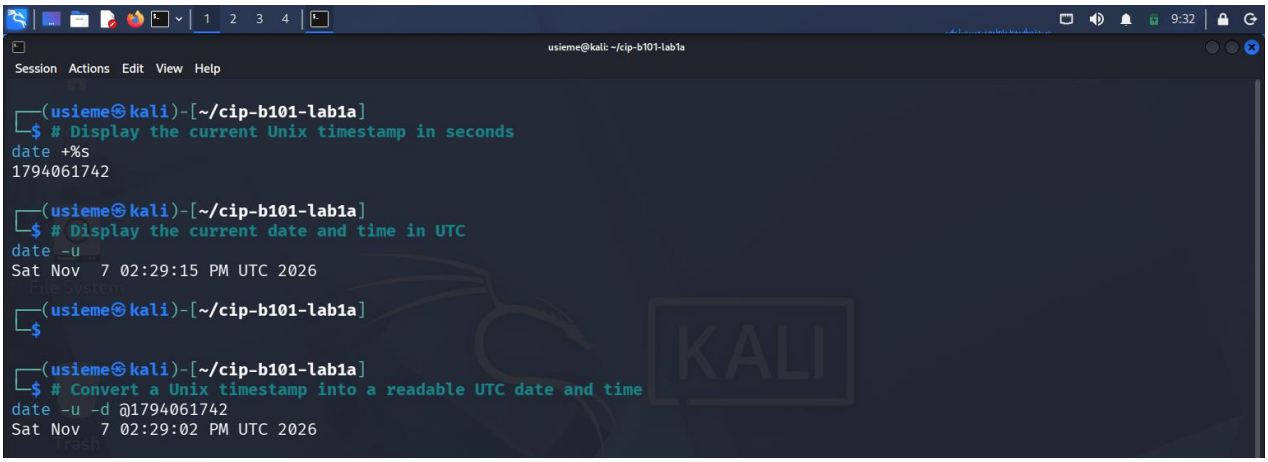
C1. Unix timestamps

Unix time counts seconds from 1 January 1970 at 00:00:00 UTC. I converted the two timestamps supplied in the lab.

Timestamp	Format	Readable UTC result
1643643491	Unix seconds	31 January 2022, 15:38:11 UTC
1675008832	Unix seconds	29 January 2023, 16:13:52 UTC
725846400	Mac Absolute Time	2 January 2024, 00:00:00 UTC
1794061742	Current Unix time	7 November 2026, 14:29:02 UTC



Screenshot 5a: Unix timestamp conversion output

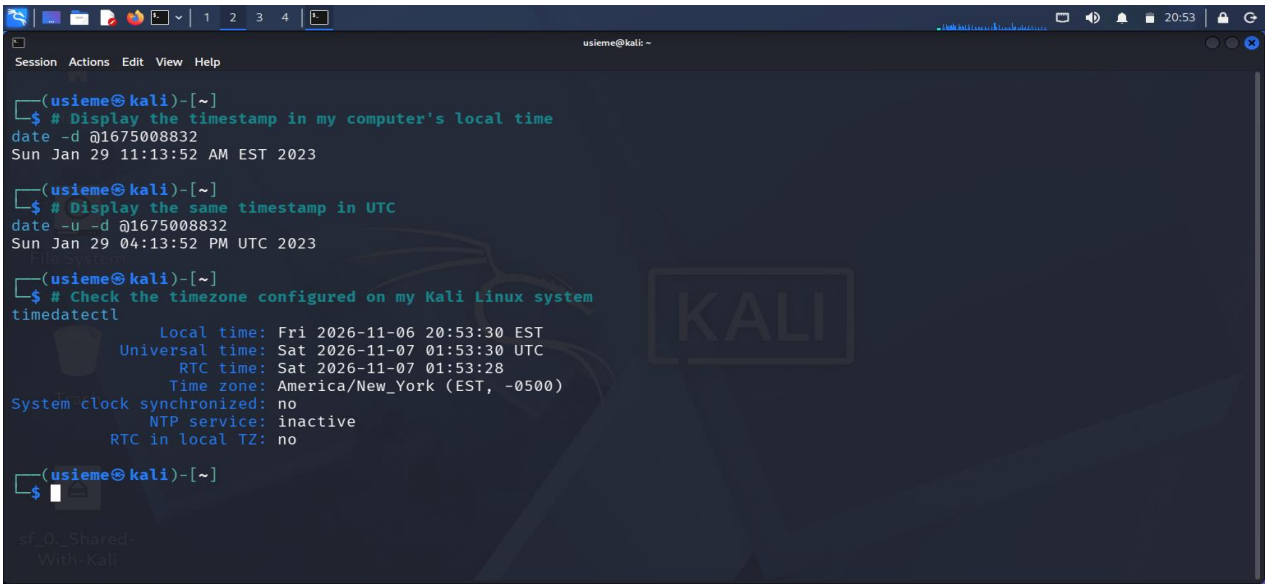


Screenshot 5b: Current Unix timestamp and UTC conversion

C2. Local time and UTC

UTC is a common time standard used to compare events across different locations. Local time depends on the computer's time zone. A forensic report should clearly state which time standard is being used.

I converted the same Unix timestamp using the date command with and without -u to compare local time and UTC. The results showed how timezone settings affect the displayed time. This is important in digital forensics when creating accurate event timelines.



Screenshot 6: Local vs UTC timestamp

C3. Mac Absolute Time

Mac Absolute Time counts seconds from 1 January 2001 at 00:00:00 UTC. The difference between the Mac and Unix starting points is 978307200 seconds.

725846400 + 978307200 = 1704153600. Converting this Unix timestamp gives 2 January 2024, 00:00:00 UTC.



Screenshot 7: Mac Absolute Time Conversion Output

Task C result: I converted the supplied Unix and Mac timestamps and stated the results in UTC.

Task D: Hashing and the Newline Problem

A hash is a fixed-length value calculated from a file's contents. It can be used to check whether a file has changed. A hash is not encryption.

D1. Comparing the Hello files

I created hello_with_newline.txt using echo and hello_without_newline.txt using echo -n. The first file contains Hello followed by a newline byte (0a). The second file contains only the bytes for Hello.

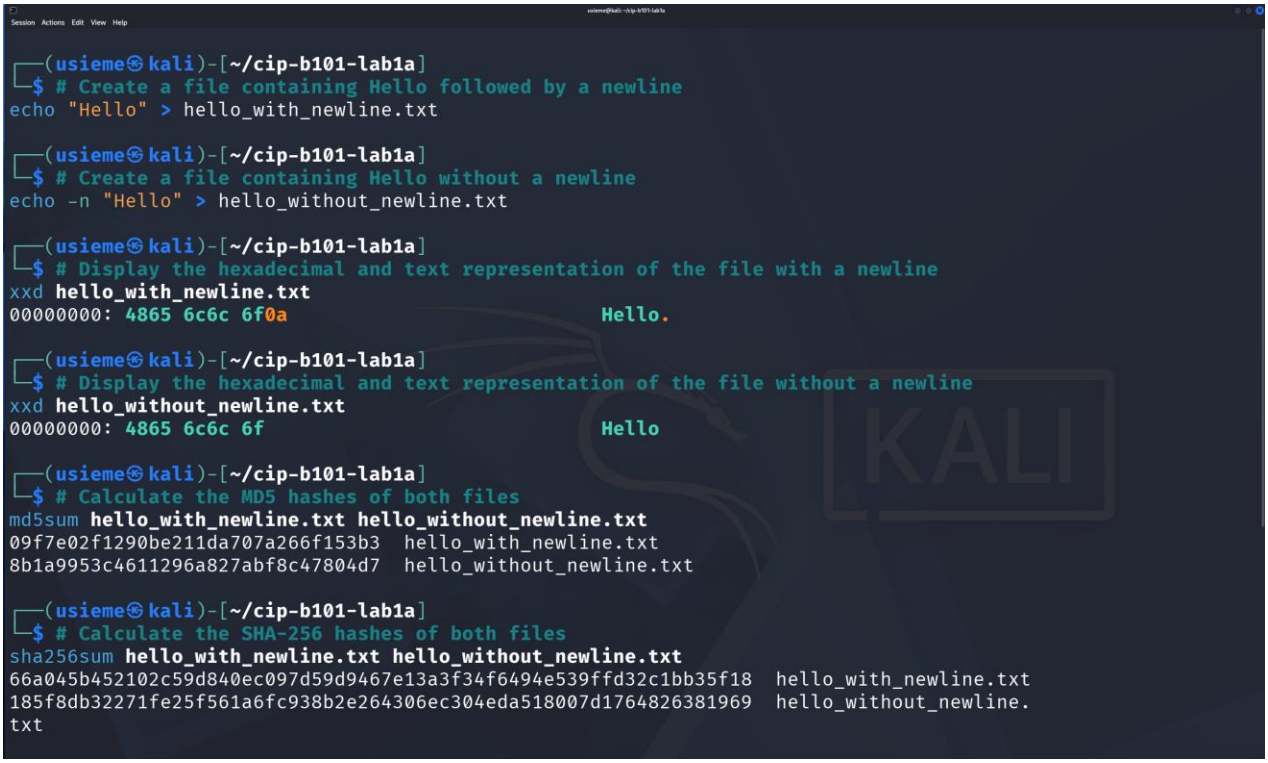
The xxd command showed these byte values:

With newline: 48656c6c6f0a

Without newline: 48656c6c6f

D2. Hash results

File	MD5 hash	SHA-256 hash
hello_with_newline.txt	09f7e2f1290be211da707a266f153b3	66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18
hello_without_newline.txt	8b1a9953c4611296a827abf8c47804d7	185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

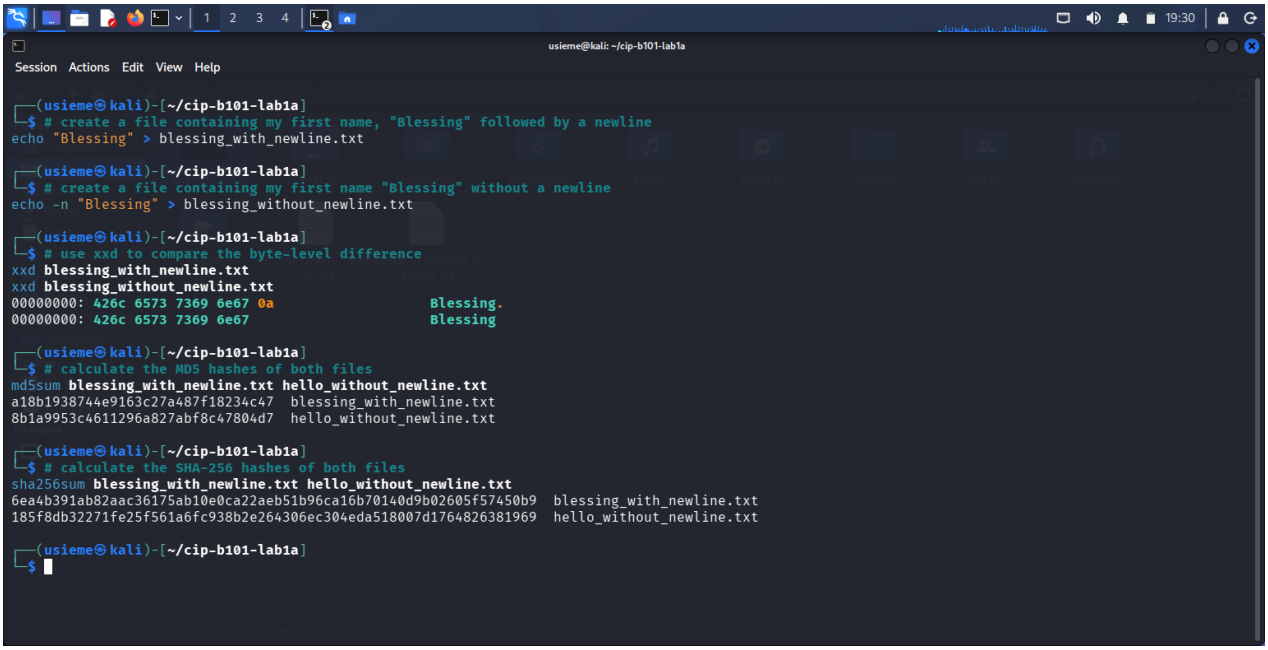


Screenshot 8a: Hash comparison using md5sum and sha256sum

D3. First-name file exercise

The brief also asks for two files containing my first name: one created with echo and one created with echo -n. These files should be compared with xxd and hashed with md5sum and sha256sum.

To create a file containing exactly my first name without an additional newline character, I would use the echo -n command before calculating its hash.



Screenshot 8b: Hash comparison using md5sum and sha256sum

- The hash values are different because the files do not contain exactly the same bytes. Even one extra newline changes the calculated hash.
- MD5 produces a 128-bit hash and SHA-256 produces a 256-bit hash. SHA-256 is preferred for modern integrity checking because MD5 is no longer considered secure against deliberate collisions.

Task D result: The comparison showed that a newline changes the file bytes and therefore changes its hash.

Task E: Endianness and Data Stored in Memory

Endianness is the order in which bytes are arranged when a value uses more than one byte. Little-endian stores the least significant byte first. Big-endian stores the most significant byte first.

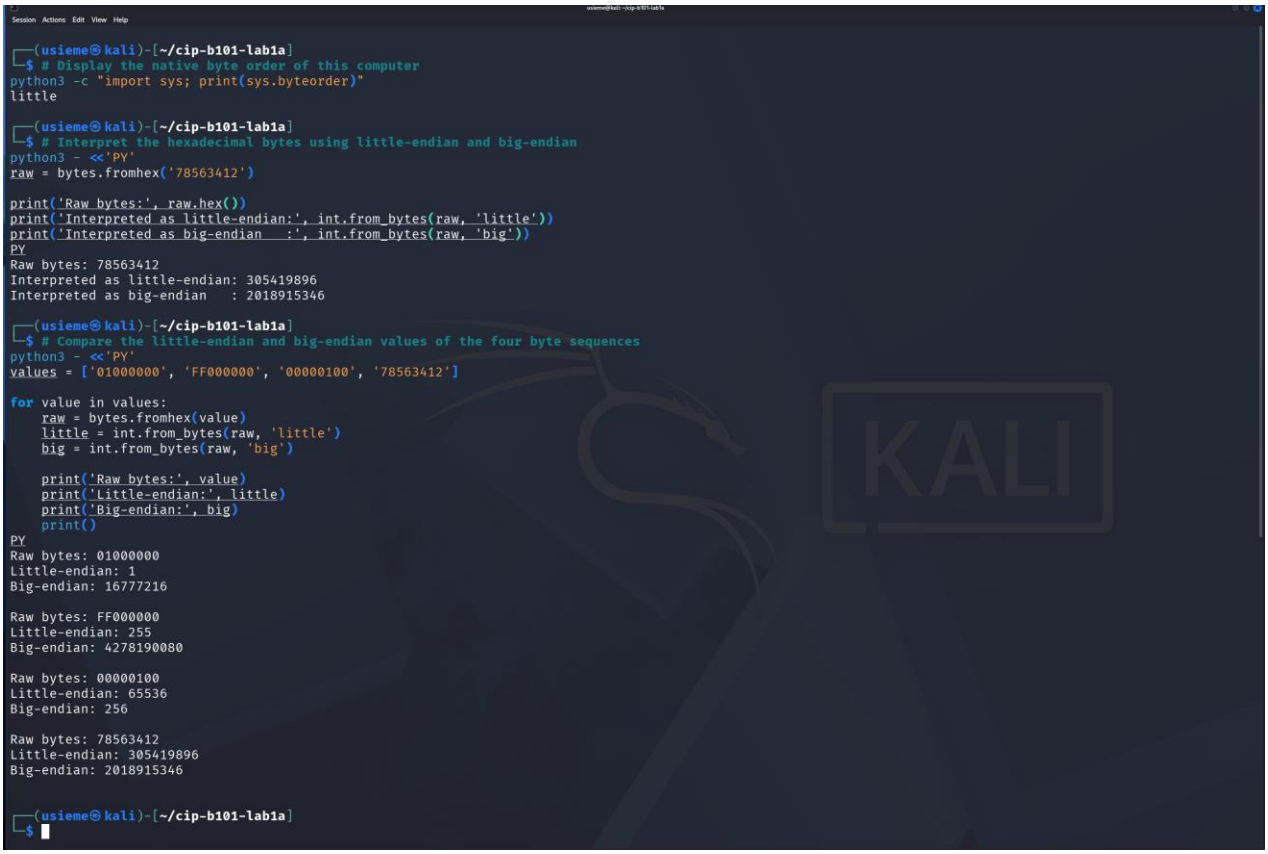
E1. System byte order

Python was used to check the system's byte order.

E2. Interpreting the same bytes

The raw bytes were 78 56 34 12. The values depend on the byte order:

Raw bytes	Little-endian decimal	Big-endian decimal
01000000	1	16777216
FF000000	255	4278190080
00000100	65536	256
78563412	305419896	2018915346



Screenshot 9: System byte oreder check and Endianess python output

E3. Why byte order matters

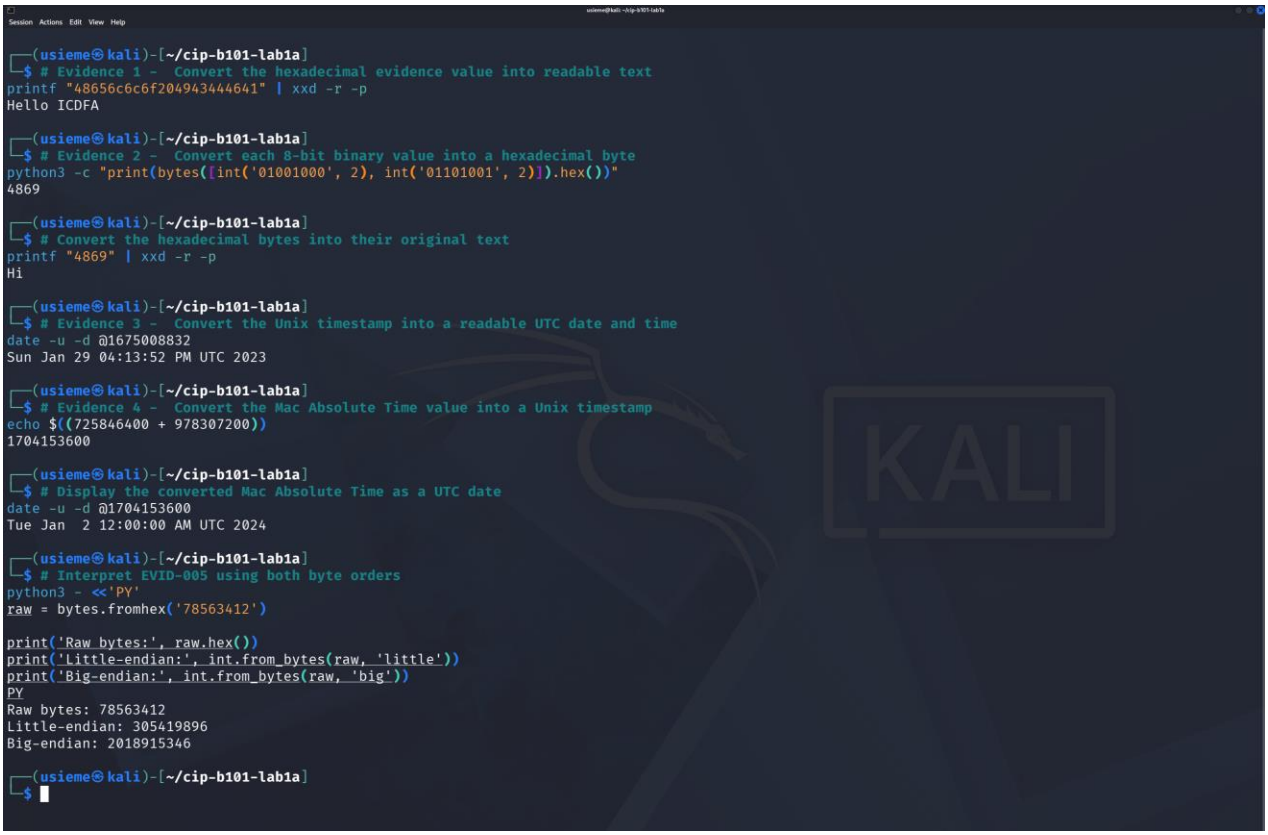
The same four bytes can produce different decimal values because the bytes are read in a different order. This matters when examining memory or disk metadata. If an examiner uses the wrong byte order, a value may be interpreted incorrectly.

Task E result: I compared little-endian and big-endian interpretations and saw how byte order changes the result.

Task F: Mini Forensic Interpretation Case

In this task, I examined five evidence values using the methods practised in the earlier tasks.

Evidence	Value	Interpretation
EVID-001	48656c6c6f204943444641	Hello ICDFA (decoded text)
EVID-002	01001000 01101001	Hi (binary characters)
EVID-003	1675008832	29 January 2023, 16:13:52 UTC (Unix time)
EVID-004	725846400	2 January 2024, 00:00:00 UTC (Mac Absolute Time)
EVID-005	78563412	Little-endian: 305419896; big-endian: 2018915346



Screenshot 10: Mini-case evidence interpretation commands

F1. What the evidence tells us

- EVID-001 and EVID-002 contain text after decoding. They are useful for text or content interpretation.
- EVID-003 and EVID-004 are timestamps. They are useful for timeline analysis because they can be converted into readable UTC dates and times.
- EVID-005 is a byte-order example. It shows that the same bytes can have different numerical meanings depending on whether they are read as little-endian or big-endian.

These values do not, by themselves, identify who created the evidence or prove that the items are connected. More information would be needed before making those conclusions.

Reflection

This lab helped me understand how computers represent information. I practised converting binary, decimal and hexadecimal values, reading text from hexadecimal bytes, converting timestamps, checking file hashes and interpreting byte order. I also learnt that small details, such as a newline or a time zone, can affect how digital evidence is understood. These skills are important in digital forensics because evidence must be interpreted accurately.